GitHup en entornos distribuidos

Presentado por; Anny Sofia Beltrán García

Presentado a; Carlos Andrés Gómez Vasco

Medicina veterinaria

Universidad agraria de Colombia

UNIAGRARIA

Ciencia de datos

2026

CAPÍTULO 5. GIT EN ENTORNOS DISTRIBUIDOS

Introducción

Git es un sistema de control de versiones distribuido que permite que diferentes personas trabajen en un mismo proyecto desde computadores independientes. Cada usuario puede tener una copia completa del repositorio, incluyendo los archivos y el historial de modificaciones. Esto facilita el trabajo colaborativo, el intercambio de información y la integración de los cambios realizados por diferentes integrantes. En este capítulo se explican los principales flujos de trabajo distribuidos, la colaboración entre repositorios, la administración de ramas y los procedimientos para compartir e integrar modificacione.

5.1. Flujos de trabajo distribuidos

Los flujos de trabajo distribuidos son métodos que permiten organizar la colaboración entre diferentes usuarios que trabajan en un mismo proyecto utilizando Git. Cada colaborador puede desarrollar sus actividades en una copia local del repositorio y posteriormente compartir sus cambios con los demás integrantes

- **Algunos flujos de trabajo comunes son:**
- Flujo de trabajo centralizado: Consiste en utilizar un repositorio remoto principal donde los colaboradores envían y reciben los cambios. Cada integrante trabaja en su copia local y comparte las modificaciones mediante comandos como **git push** y **git pull.**
- Flujo de trabajo del administrador de integración: En este modelo existe una persona encargada de revisar e integrar los cambios enviados por los colaboradores. Los integrantes desarrollan sus actividades de manera independiente y posteriormente envían sus propuestas para que sean revisadas

- Flujo de trabajo con múltiples repositorios: Permite que diferentes colaboradores mantengan sus propios repositorios y compartan modificaciones mediante solicitudes de integración o intercambio de cambios. Es utilizado en proyectos donde participan diferentes equipos o desarrolladores. Estos modelos permiten organizar el trabajo según las características del proyecto y la cantidad de personas involucradas

5.2. Contribuir a un proyecto

Contribuir a un proyecto significa participar en su desarrollo mediante la creación, modificación o corrección de archivos y funcionalidades. En Git, los colaboradores pueden realizar cambios en sus repositorios locales y posteriormente compartirlos con el repositorio principal

- **Para contribuir a un proyecto se pueden seguir estos pasos:**

1. Obtener una copia del repositorio mediante el comando: **git clone URL_DEL_REPOSITORIO**
2. Ingresar a la carpeta del proyecto
3. Crear una rama para trabajar de manera independiente utilizando: **git switch -c nombre_de_la_rama**
4. Realizar las modificaciones necesarias en los archivos
5. Agregar los cambios mediante: **git add**
6. Registrar las modificaciones con: **git commit -m "Descripción de los cambios"**
7. Enviar la rama al repositorio remoto mediante: **git push -u origin nombre_de_la_rama**

Después de realizar estos pasos, los cambios pueden ser revisados e integrados al proyecto principal

5.3. Mantener un proyecto

Mantener un proyecto consiste en administrar sus modificaciones, revisar las contribuciones de los colaboradores y conservar el repositorio organizado. Esta actividad es importante para evitar errores, resolver problemas y garantizar que los cambios incorporados sean adecuados para el funcionamiento del proyecto

- **Algunas actividades importantes para mantener un proyecto son:**
- Revisar las contribuciones: Consiste en analizar las modificaciones propuestas por los colaboradores antes de integrarlas al proyecto principal
- Administrar las ramas: Permite organizar las diferentes líneas de desarrollo y separar las funcionalidades nuevas de la versión estable
- Corregir errores: Consiste en identificar y solucionar problemas encontrados durante el desarrollo del proyecto
- Actualizar el repositorio: Permite incorporar los cambios realizados por otros colaboradores y mantener sincronizadas las versiones del proyecto
- Organizar el historial: Facilita la consulta de las modificaciones realizadas y permite identificar quién realizó cada cambio

5.4. Etiquetado de las versiones

El etiquetado permite identificar puntos importantes dentro del historial de un repositorio. Se utiliza principalmente para señalar versiones específicas de un proyecto, como una entrega inicial, una actualización o una versión final

- **Git permite crear diferentes tipos de etiquetas:**
- Etiquetas ligeras: Son referencias sencillas que identifican una confirmación específica del historial

- Para crear una etiqueta ligera se utiliza: **git tag v1.0**
- Etiquetas anotadas: Almacenan información adicional, como el nombre del autor, la fecha y un mensaje descriptivo
- Para crear una etiqueta anotada se utiliza: **git tag -a v1.0 -m "Primera versión del proyecto"**
- Consultar las etiquetas existentes: Se utiliza el comando **git tag**
- Enviar una etiqueta al repositorio remoto: Se utiliza **git push origin v1.0**
- Enviar todas las etiquetas: Se utiliza **git push origin --tags**

Las etiquetas permiten organizar las versiones del proyecto y facilitan la identificación de entregas importantes

5.5. Los alias de Git

Los alias son nombres cortos que se pueden configurar para ejecutar comandos de Git de manera más rápida y sencilla. Su utilización permite ahorrar tiempo cuando se trabaja frecuentemente con comandos largos

- **Por ejemplo:**
- Crear un alias para consultar el historial resumido: **git config --global alias.hist "log --oneline"**
- Crear un alias para consultar el estado del repositorio**: git config --global alias.st status**

Después de configurar estos alias, se pueden ejecutar los comandos **git hist** y **git st** para realizar las mismas operaciones. Los alias son opcionales y cada usuario puede personalizarlos de acuerdo con sus necesidades

5.6. Sincronización del trabajo entre colaboradores

La sincronización permite mantener actualizadas las copias locales y remotas de un proyecto. Es fundamental cuando varias personas realizan modificaciones sobre un mismo repositorio. Git proporciona diferentes comandos para realizar esta tarea:

- Descargar e integrar cambios: Se utiliza **git pull** para obtener las modificaciones del repositorio remoto e integrarlas en la rama local actual
- Descargar información sin integrar los cambios: Se utiliza **git fetch** para obtener las actualizaciones del repositorio remoto sin modificar directamente la rama de trabajo
- Enviar modificaciones: Se utiliza **git push** para compartir los commits locales con el repositorio remoto
- Consultar el estado del proyecto: Se utiliza **git status** para revisar los archivos modificados y los cambios pendientes de registrar

Estos comandos permiten mantener la información actualizada y facilitan el intercambio de modificaciones entre los colaboradores

5.7. Ejercicio práctico: trabajo colaborativo con Git

Permite comprender cómo un colaborador puede obtener un repositorio, realizar modificaciones y compartir sus cambios mediante Git

- **Procedimiento:**

1. Abrir Git Bash o CMD
2. Obtener una copia del repositorio utilizando: **git clone URL_DEL_REPOSITORIO**
3. Ingresar a la carpeta del proyecto
4. Crear una rama nueva utilizando: **git switch -c colaboracion**
5. Crear o modificar un archivo dentro del proyecto

6. Agregar los cambios mediante: **git add**
7. Registrar las modificaciones con: **git commit -m "Realiza cambios de colaboración"**
8. Enviar la rama al repositorio remoto mediante: **git push -u origin colaboracion**
9. Consultar las ramas disponibles utilizando: **git branch -a**
10. Revisar los cambios enviados en la plataforma GitHub

Se quiere saber cómo se obtiene un repositorio, se realizan modificaciones de manera independiente y se comparten los cambios con otros colaboradores mediante Git

Conclusiones

Git en entornos distribuidos permite que diferentes personas participen en un mismo proyecto desde sus propios computadores, manteniendo copias independientes del repositorio y compartiendo las modificaciones realizadas. Los flujos de trabajo distribuidos facilitan la organización de las actividades, mientras que los comandos git clone, git pull, git push y git fetch permiten intercambiar información entre repositorios locales y remotos. Además, el uso de ramas, etiquetas y alias contribuye a mantener los proyectos organizados y facilita el seguimiento de las modificaciones. El trabajo con Git en entornos distribuidos es fundamental para desarrollar proyectos colaborativos, porque permite compartir información, organizar las contribuciones y administrar los cambios de manera eficiente

Referencias bibliográficas

1. Chacon, S., & Straub, B. (2014). Pro Git (2.ª ed.). Apress. https://git-scm.com/book/es/v2

2. Git. (s. f.). Git en entornos distribuidos. https://git-scm.com/book/es/v2/Git-en-entornos-distribuidos
3. Git. (s. f.). Flujos de trabajo distribuidos. https://git-scm.com/book/es/v2/Git-en-entornos-distribuidos-Flujos-de-trabajo-distribuidos
4. Git. (s. f.). Contribuir a un proyecto. https://git-scm.com/book/es/v2/Git-en-entornos-distribuidos-Contribuir-a-un-proyecto
5. Git. (s. f.). Mantener un proyecto. https://git-scm.com/book/es/v2/Git-en-entornos-distribuidos-Mantener-un-proyecto